

Configurando Conexões de Banco de Dados no Airflow

Tutorial Prático: SQLite e PostgreSQL

Disciplina de Extração de Dados

IBMEC

14 de maio de 2026

- Aprender a cadastrar credenciais de forma segura no **Apache Airflow**.
- Compreender o uso da interface Web (**Admin** → **Connections**).
- Configurar uma conexão para testes rápidos (**SQLite**).
- Configurar uma conexão de nível de produção (**PostgreSQL**).

Segurança em Primeiro Lugar: O Cofre do Airflow

- **Regra de Ouro:** Nunca coloque senhas, usuários ou IPs (hardcoding) no código Python das suas DAGs!
- O Airflow possui um cofre interno criptografado chamado **Connections**.
- No código, você escreve apenas o nome do **identificador** (o `conn_id`).
- O *Hook* do Airflow vai até o cofre, pega as credenciais criptografadas, e abre a porta do banco de dados para você.

Cenário 1: SQLite (Banco de Dados Local)

- O SQLite é um banco de dados que não precisa de servidor. Ele funciona em um simples arquivo (.db).
- **Pré-requisito (Provider):** `pip install apache-airflow-providers-sqlite`
- **Passo a Passo na Interface Web:**
 - 1 Clique em **Admin** → **Connections**.
 - 2 Clique no botão + (Add a new record).
 - 3 **Connection Id:** `sqlite_default` (Usado no código Python).
 - 4 **Connection Type:** SQLite.
 - 5 **Host:** `/tmp/meu_banco_airflow.db` (Ou `C:/Temp/banco.db` no Windows).
 - 6 Deixe Login e Password **em branco** (SQLite não tem usuário).
 - 7 Clique em **Save**.

Testando a Conexão: SQLite

No código da sua DAG, você referencia a configuração recém-criada através da propriedade `conn_id`.

```
1 from airflow.providers.common.sql.operators.sql import SQLExecuteQueryOperator
2
3 # A magia acontece aqui: o Airflow lê o 'sqlite_default' do cofre
4 tarefa_sqlite = SQLExecuteQueryOperator(
5     task_id='teste_sqlite',
6     conn_id='sqlite_default',
7     sql="SELECT 1;"
8 )
```

Cenário 2: PostgreSQL (Padrão de Mercado)

- O PostgreSQL é um servidor relacional robusto.
- **Pré-requisito (Provider):** `pip install apache-airflow-providers-postgres`
- **Passo a Passo na Interface Web:**
 - 1 Clique em **Admin** → **Connections** → +.
 - 2 **Connection Id:** `postgres_default`.
 - 3 **Connection Type:** Postgres.
 - 4 **Host:** `localhost` ou `host.docker.internal` (Se Airflow via Docker e DB via Windows).
 - 5 **Schema:** `postgres` (Ou o nome do banco criado).
 - 6 **Login:** `postgres` (Seu usuário).
 - 7 **Password:** Sua Senha e **Port:** 5432.
 - 8 Clique em **Save**.

Testando a Conexão: PostgreSQL

O código para qualquer banco de dados é virtualmente idêntico graças ao `SQLExecuteQueryOperator`. O que muda é apenas o cofre apontado!

```
1 from airflow.providers.common.sql.operators.sql import SQLExecuteQueryOperator
2
3 # O mesmo operador, apenas mudando o conn_id!
4 tarefa_postgres = SQLExecuteQueryOperator(
5     task_id='teste_postgres',
6     conn_id='postgres_default',
7     sql="SELECT 1;"
8 )
```

O Tipo de Banco não aparece?

Se ao cadastrar, a opção "Postgres" ou "SQLite" não aparecer no **Connection Type**, significa que o *Provider* não está instalado no seu ambiente. **Solução:** Rode `pip install ...` no terminal do Airflow e reinicie a aplicação.

Connection Refused (Postgres)

Isso significa que o Airflow não encontrou o banco. Verifique: 1) O banco está ligado? 2) A porta 5432 está correta? 3) Se usando Docker: localhost dentro do container não enxerga seu PC físico. Tente usar o IP da máquina ou `host.docker.internal`.